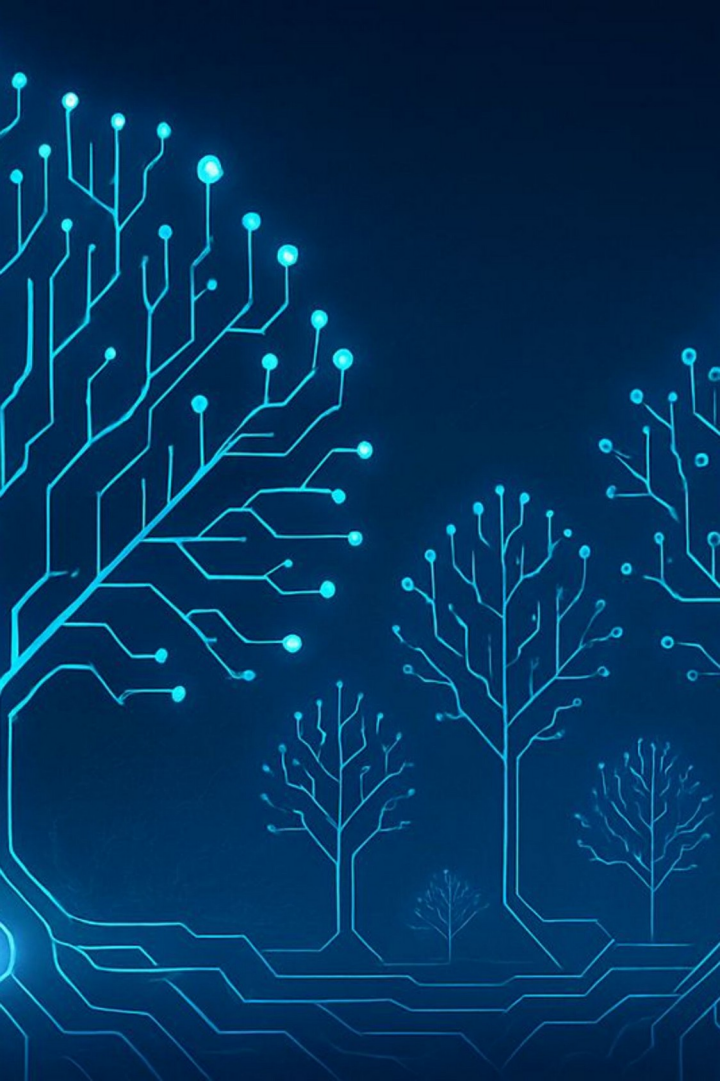


An abstract graphic on the left side of the slide. It features several stylized trees whose branches are composed of glowing blue circuit traces. Small blue dots, resembling solder points or data nodes, are placed at the ends of many branches. The background is a dark blue gradient.

Árbol de Sintaxis Abstracta (AST)

Representación jerárquica de la estructura sintáctica del código fuente. Elimina detalles redundantes —paréntesis, puntos y coma— y presenta la lógica del programa de forma limpia para las fases posteriores del compilador.

Integrante 1 - Pregunta 1,2,3

Características Principales del AST

Jerarquía Lógica

Nodos internos: operadores y estructuras de control. Nodos hoja: operandos y valores literales.

Simplicidad

Facilita el análisis semántico y la generación de código, libre de ruido gramatical.

Independencia del Formato

Espacios en blanco o líneas adicionales no alteran la estructura del AST.

Soporte Semántico

Permite verificar tipos y detectar variables no declaradas.

Optimización

Plegado de constantes ($x = 5 * 2$ lto 10) y eliminación de código muerto.

Desacoplamiento

Un mismo AST genera código para arquitecturas x86 o ARM sin reanalizar la sintaxis.

Ejemplos de Entrada y Representación

Ejemplo A: Operación Aritmética

Entrada: $3 + 5 * 2$

La multiplicación tiene mayor prioridad y se agrupa en un nivel inferior:

```
+
├ 3
└ *
  ├── 5
  └ 2
```

Ejemplo B: Asignación de Variable

Entrada: $x = 10$

El operador de asignación actúa como raíz:

```
=
├ x
└ 10
```

Implementación en Python

Nodo base y construcción del árbol para $3 + 5 * 2$:

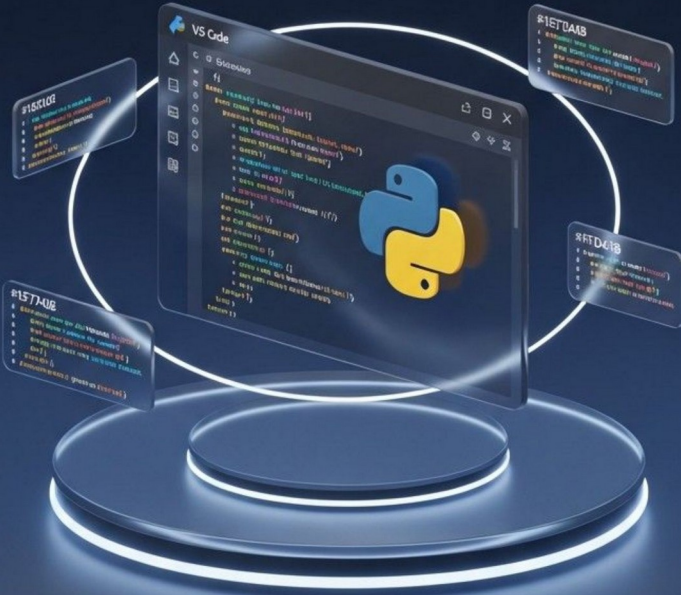
```
class NodoAST:
    def __init__(self, tipo_token, valor, izquierdo=None, derecho=None):
        self.tipo_token = tipo_token # OPERADOR, NUMERO, VARIABLE
        self.valor = valor # Lexema real (+, *, 3, etc.)
        self.izquierdo = izquierdo
        self.derecho = derecho

# Construcción: 3 + 5 * 2
num3 = NodoAST("NUMERO", "3")
num5 = NodoAST("NUMERO", "5")
num2 = NodoAST("NUMERO", "2")
nodo_mult = NodoAST("OPERADOR", "*", num5, num2) # Subárbol prioritario
raiz_ast = NodoAST("OPERADOR", "+", num3, nodo_mult) # Raíz
```

Comparación: LL vs LR

Criterio	LL — Descendente	LR — Ascendente
Dirección	Derivación por la izquierda	Derivación inversa por la derecha
Enfoque	Desde el símbolo inicial hacia los tokens	Desde los tokens hacia el símbolo inicial
Complejidad	Baja: sencillo de implementar manualmente	Alta: requiere tablas de transición de estados
Flexibilidad	Restringido: no admite ambigüedades ni recursión izquierda	General: maneja gramáticas complejas y expresiones anidadas

Evidencia de Uso Actual



Análisis LL

Compiladores de **Python** y **Java** usan variantes avanzadas de LL (parsers recursivos descendentes PEG o LL(k)), facilitando el control manual y la personalización de mensajes de error.

Análisis LR

Herramientas de **GCC** para C/C++ han dependido históricamente de motores LALR(1) (subconjunto eficiente de LR) generados de forma automatizada.

Implementación Comparativa: Parser LL

Parser recursivo descendente manual para el lenguaje *SL* (regla: *INT + INT*):

```
class ParserLL1Manual:
    def __init__(self, lista_tokens):
        self.tokens = lista_tokens
        self.indice = 0

    def emparejar(self, token_esperado):
        token_actual = self.tokens[self.indice]
        if token_actual == token_esperado:
            print(f"[OK] Token emparejado: {token_esperado}")
            self.indice += 1
        else:
            raise SyntaxError(f"Se esperaba '{token_esperado}'")

    def analizar(self):
        self.emparejar("TK_INT")
        self.emparejar("TK_PLUS")
        self.emparejar("TK_INT")
```

Parser LR: Simulación Shift / Reduce

Maneja una pila de estados y tokens, realizando acciones de **desplazamiento** (shift) o **reducción** (reduce) hasta alcanzar el símbolo inicial:

```
class ParserLRSimulado:
    def __init__(self, lista_tokens):
        self.tokens = lista_tokens + ["$"]
        self.pila_estados = [0]
        self.pila_simbolos = []

    def analizar(self):
        # SHIFT: desplazar tokens a la pila
        self.pila_simbolos.append("TK_INT")
        self.pila_estados.append(1)
        # REDUCE: reducir a la regla base
        self.pila_simbolos = [""]
```


Generadores de Analizadores Sintácticos

Metacompiladores que toman una gramática formal (BNF) y generan automáticamente el código de un lexer y un parser funcional.

Bison

Generador clásico del proyecto GNU, compatible con reglas LALR(1). Usado junto a Flex en entornos C/C++.

ANTLR

Metacompilador moderno que genera parsers ALL(*) y exporta código a múltiples lenguajes: Java, Python, C++ y C#.



Estrategias de Gestión de Errores

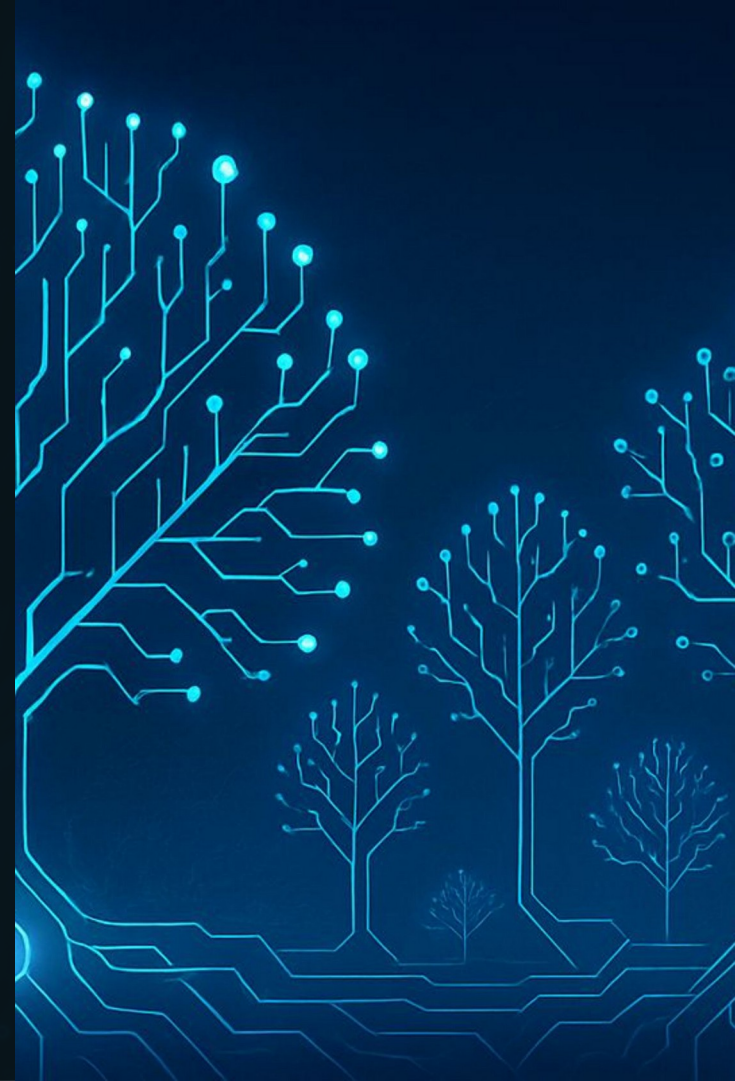
Cuando los tokens violan la gramática, un parser robusto aplica técnicas de contención para seguir evaluando el resto del archivo:

- **Modo Pánico:** Descarta tokens hasta un delimitador seguro (;, }, \n) y reanuda el análisis.
- **Inserción/Borrado:** Inyecta virtualmente el token faltante (ej: paréntesis de cierre) y registra advertencia.
- **Recuperación a Nivel de Frase:** Microcirugía local reemplazando tokens erróneos (coma por punto y coma).
- **Producciones de Error:** Reglas que capturan errores comunes (ej: if sin then) y reportan advertencias sin colapsar.

Análisis Sintáctico

Definiciones y conceptos fundamentales del análisis sintáctico en compiladores.

Integrante 2 – Arquitecto de la Gramática Docker — Pregunta 4-A



Componentes del Análisis

Analizador Léxico (Lexer)

Tokeniza cadenas de caracteres (ej. `if` → `TK_IF`), abstrayendo detalles léxicos para la siguiente fase.

Analizador Sintáctico (Parser)

Núcleo del análisis: consume tokens, valida reglas gramaticales y genera estructuras jerárquicas.

Gramática Libre de Contexto (GLC)

Punto de partida teórico que describe las reglas sintácticas de un lenguaje formal.

Árbol de Sintaxis Abstracta (AST)

Estructura jerárquica que representa la lógica validada del programa.

Clasificación de Analizadores Sintácticos

Parsers LL

Left-to-right, Leftmost derivation

- Baja complejidad: fáciles de construir manualmente (parsers recursivos descendentes)
- Altamente restrictivos: gramáticas no ambiguas, sin recursión por la izquierda

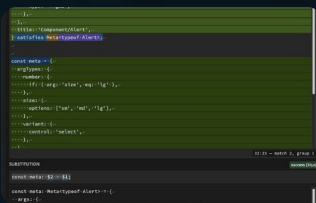
Parsers LR

Left-to-right, Rightmost derivation

- Alta complejidad: requieren tablas grandes
- Alta flexibilidad: manejan gramáticas generales y expresiones anidadas complejas

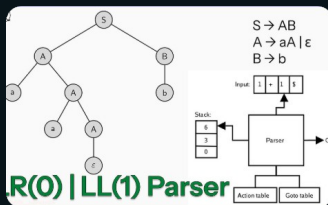
Metacompiladores y Herramientas

Herramientas que procesan descripciones gramaticales para construir automáticamente parsers, lexers y traductores.



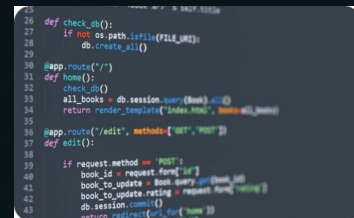
Flex

Genera analizadores léxicos basándose en expresiones regulares (regex).



Bison

Genera parsers del tipo **LALR(1)**, subconjunto de la familia LR.



ANTLR

Metacompilador versátil que soporta generación de código para **Java, Python y C++**.

Manejo de Errores y Técnicas Híbridas

Recuperación Tradicional

Técnicas como el **modo panic** o la inserción de tokens faltantes para mantener el análisis en marcha.

Fallback a IA

Si un token falla o es ambiguo, un **LLM** sugiere correcciones para continuar el proceso.

Umbral de Confianza

Marca cuándo una coincidencia de tokens es suficiente o debe revisarse con ayuda externa.

Experimento de Carga Multi-lenguaje

Comparación de parsers Python, Java y C sobre la gramática de redes Docker.

Integrante 3 - Ingeniero de Rendimiento Multi-lenguaje — Pregunta 4-B



Metodología del Experimento

Cómo se diseñó la comparación para que fuera justa entre los tres lenguajes.

Misma Gramática (GLC)

Se reutiliza DockerNetworks.g4 del Arquitecto de la Gramática para los tres parsers: comparación válida sobre el mismo lenguaje L.

Tres Implementaciones

Python y Java generados con ANTLR 4.13.1; C implementado a mano con Flex + Bison replicando las mismas reglas léxicas y su orden.

Dataset de Prueba

10 archivos docker-compose ($n=10$, dentro del rango $5 < n < 20$ exigido), generados por el Arquitecto de la Gramática.

Medición en Memoria

200 repeticiones por archivo y lenguaje; se descarta la primera repetición para evitar el sesgo de arranque en frío (JIT / intérprete).

Tres Implementaciones, un mismo Lenguaje L

Mismo punto de partida (la GLC); tres metacompiladores y lenguajes distintos.



Python (ANTLR)

Generado con `-Dlanguage=Python3`.
Runtime `antlr4-python3-runtime`.
Medición con `time.perf_counter()`.



Java (ANTLR)

Generado con `-Dlanguage=Java`.
Compilado con `antlr-4.13.1-complete.jar`
(metacompilador + runtime). Medición
con `System.nanoTime()`.

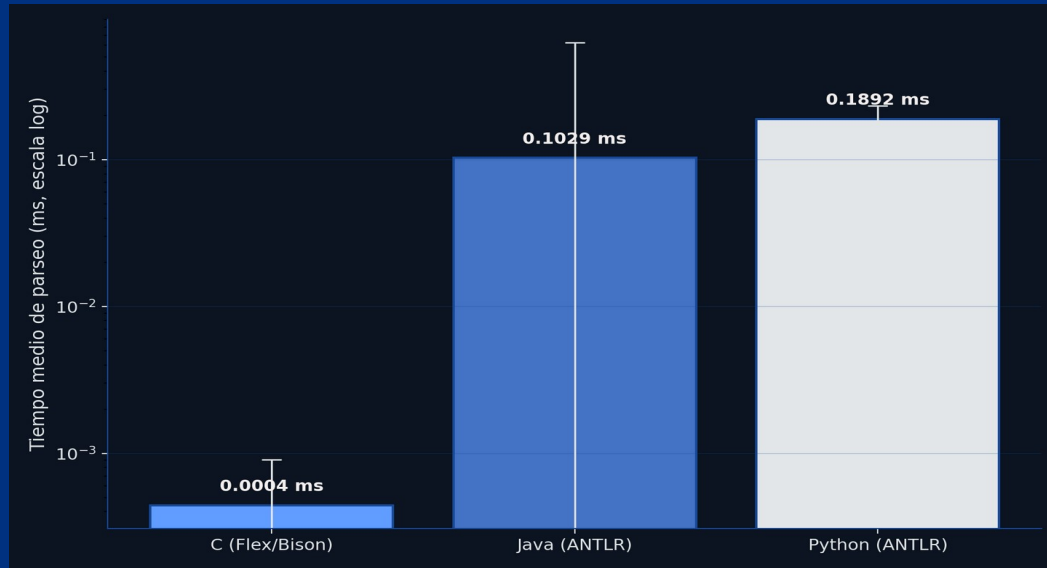


C (Flex + Bison)

Implementación manual —no generada
por ANTLR— que replica el mismo orden
de reglas léxicas. Medición con
`clock_gettime(CLOCK_MONOTONIC)`.

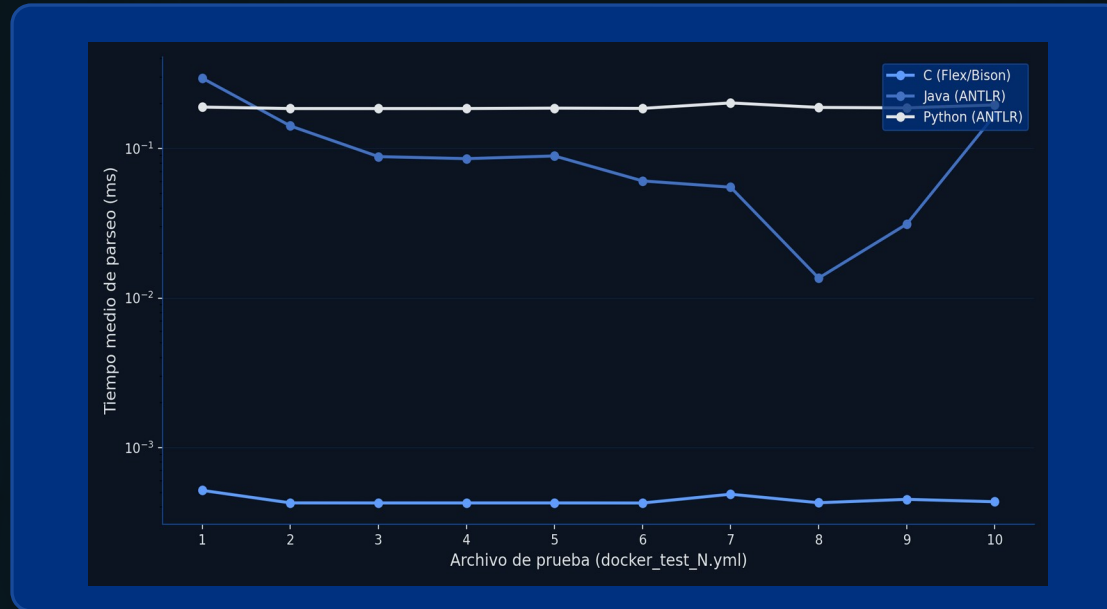
Resultados: Tiempo Medio por Lenguaje

10 archivos × 199 repeticiones útiles por lenguaje (5 970 mediciones en total).



Resultados por Archivo de Prueba

El orden de magnitud se mantiene estable en los 10 archivos, sin importar su tamaño.



Hallazgo Clave: Ambigüedad Léxica

En la GLC compartida, `ID` se declara antes que `STRING`. Por máxima coincidencia con desempate por orden de declaración, un texto como `bridge` siempre se tokeniza como `ID`, nunca coincide con la producción `DRIVER_KEY COLON STRING`.

Confirmación empírica

Los tres parsers (Python, Java, C) rechazan el 100% de los archivos, de forma idéntica y reproducible en los tres lenguajes → el problema está en la especificación de la gramática, no en ninguna de las tres implementaciones.

Discusión y Trabajo Futuro

Nativo vs. JVM vs. Intérprete

C es ~230x más rápido que Java y ~430x más rápido que Python en promedio: coincide con la teoría (código nativo vs JIT vs interpretación pura).

Varianza del JIT

Java muestra una desviación estándar mayor que su propia media, por el calentamiento del compilador JIT de HotSpot; la mediana es más representativa que el promedio.

Trabajo Futuro

Generar un cuarto parser en C con ANTLR (-Dlanguage=C++) permitiría aislar la ventaja del lenguaje de la del generador de parsers.

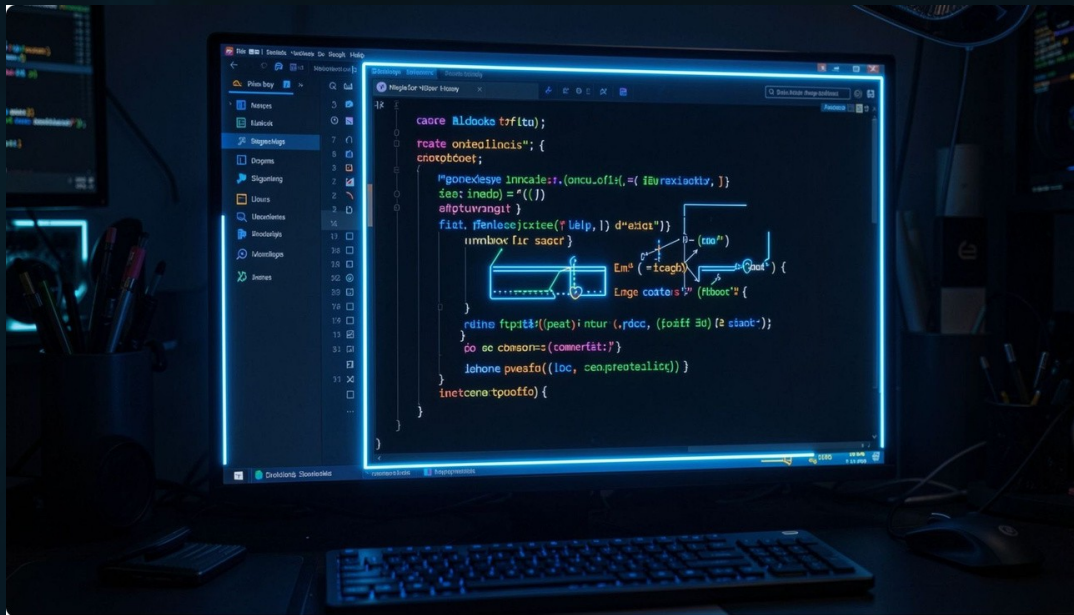


Entorno Híbrido de Compilación

Implementación de un compilador moderno para **UnegScript** que combina autocorrección basada en similitud y respaldo a Inteligencia Artificial (LLM) cuando los errores superan los umbrales tolerables.

INTEGRANTE 4 · UNEGSCRIPT -

Fundamentos del Sistema



Dos pilares en Python

Algoritmo de Similitud (difflib)

Emula la Distancia de Levenshtein, midiendo en escala **0.0–1.0** la similitud entre palabras mal escritas y palabras clave legítimas (var, imprimir, funcion, si, sino).

Umbrales de Confianza

≥ 0.8 → autocorrección silenciosa. < 0.8 → error crítico, se activa el fallback a la IA.



Componentes del Pipeline

consultar_llm

Puente con APIs de IA (OpenAI, Gemini, DeepSeek).
Recibe tipo de error, contexto y código afectado; devuelve recomendaciones inteligentes para corregir UneyScript.

analizar_lexico

El Lexer tokeniza el código. Si encuentra un término desconocido, calcula similitud: corrige en tiempo real si ≥ 0.8 o deriva al LLM si < 0.8 .

analizar_sintaxis

Parser recursivo descendente que valida estructuras (ej. `var [nombre] = [valor]`). Intercepta excepciones y consulta al LLM para guiar al usuario.

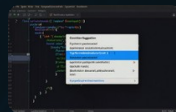
Escenarios de Demostración

Cuatro pruebas prácticas para la defensa del proyecto:



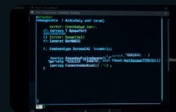
Código Correcto

Valida el funcionamiento estándar del compilador bajo sintaxis limpia.



Error Leve (Autocorrección)

imprimir $\rightarrow$ similitud ≥ 0.8
 $\rightarrow$ corrección automática.



Error Léxico Crítico

imprzxt $\rightarrow$ confianza < 0.8
 $\rightarrow$ respuesta del LLM en terminal.



Error Sintáctico

Omisión de = $\rightarrow$ parser detecta fallo estructural y solicita asistencia a la IA.

Resumen del Sistema



Valor diferencial

UnegScript demuestra cómo un compilador moderno puede **interactuar en tiempo real con modelos de lenguaje** para asistir al programador frente a fallas léxicas y sintácticas, superando el enfoque tradicional de simplemente rechazar el código.



Componente más avanzado de la defensa: integración en tiempo real con LLM.